

VS Moradia

Ata de Reunião sobre View Materializada PRT Propostas – 04/04

Participantes

Rafael Schwenck (rafael.schwenck@bradesco.com.br)
Alex Silva (alexl.silva@bradesco.com.br)
Felipe Souza (felipec.souza@bradesco.com.br)
Andreza (andreza.inforzato@bradesco.com.br)
Luciana Lima (luciana.lima@bradesco.com.br)
Rodrigo Santos (rodrigo.ad.santos@bradesco.com.br)
Ana Shopping (ana.p.shopping@bradesco.com.br)
Alessandro Lyra (alessandro.l.lyra@bradesco.com.br)

Contexto e Problema

A criação da View Materializada visava realizar consultas na tabela PRT_Proposta no Oracle (CRIM) utilizando métodos de Refresh (FAST Refresh) para otimizar consultas complexas sem impactar o transacional.

Impacto

No dia da execução da query da View Materializada, iniciou-se um incidente no banco de dados, causando locks e impactando a malha de produção. A query foi iniciada em 29/03, aproximadamente às 23:00.

Solução para o incidente: Foi realizado o comando DROP MATERIALIZED VIEW às 00:00.

Discussões

Alex Silva: Identificou que a query estava sendo executada com REFRESH FAST ON COMMIT, que é por transação. Exemplo: Quando um UPDATE de 5 milhões de linhas é feito, o comando espera o término do UPDATE e inicia a execução toda de uma vez. Sugere usar um REFRESH minuto a minuto.

Felipe Souza: Questionou a necessidade do uso da View Materializada em tempo real.

Ana Shopping: Explicou que o cliente faz uma amortização e não pode iniciar outra antes de finalizar a anterior. Caso ocorra algum erro, o cliente poderia iniciar várias requisições ou enviar propostas. E a base deve ser atualizada para evitar retiradas indevidas de valores da conta do cliente.

Felipe Souza: Argumentou que o controle deveria ser feito na aplicação, não replicando a tabela.

Ana Shopping: Concordou, mas destacou a necessidade de garantir a integridade da informação. Quando uma amortização é realizada, um evento é criado na aplicação, impedindo novas requisições enquanto a anterior está em andamento.

Felipe Souza: Perguntou para onde estavam replicando a tabela.

Andreza: Esclareceu que não estava ocorrendo replicação, mas sim um SELECT na View Materializada.

Felipe Souza: Argumentou que a View estava replicando uma tabela de 13GB e outra de 2.5GB apenas para consulta e controle. Sugeriu fazer a query diretamente na tabela.

Andreza: Explicou que a necessidade era atender o time do ID que consome dados de uma tabela inteira.

Rafael Schwenck: Destacou que a View evita acessar diretamente o transacional e bloquear o transacional.

Felipe Souza: Argumentou que a execução da View cria uma trigger e outra tabela, onerando a base de dados da mesma forma.

Rafael Schwenck: Observou que a execução implica no aumento da transação original, tornando-a mais lenta. Usar o commit faz a mesma operação linha a linha dentro da mesma transação, executando dois inserts simultaneamente.

Felipe Souza: Destacou que a execução em tempo real consome mais memória e log. Sugeriu agendar a execução para desonerar o processamento.

Rafael Schwenck: Explicou que o ID realiza muitas consultas, onerando o processador, mas não gerando locks em tabelas. A query consome memória e processamento.

Andreza: Sugeriu fazer um SELECT direto na tabela (PRT_PROPSTA), realizando um teste de carga para analisar possíveis bloqueios e garantir o uso correto das chaves das tabelas. Propôs tratar a questão do ID separadamente, usando uma trigger.

Felipe Souza: Sugeriu fazer ingestão para o Data Lake para processamentos separados, via arquivo ou uma vez por dia.

Andreza: Informou que o ID está fazendo via arquivo, por XFB para o Lake, desonerando a consulta. A segunda parte será fazer o DELTA e uma trigger.

Conclusão e Recomendação para a Vila Moradia

Conclusão: A reunião destacou a necessidade de otimizar consultas complexas sem impactar o transacional, utilizando a View Materializada. No entanto, a execução da View causou incidentes no banco de dados, gerando locks e impactando a produção. A solução imediata foi remover a View Materializada. Discussões apontaram alternativas para melhorar a performance e garantir a integridade dos dados, como realizar consultas diretamente na tabela e usar triggers para atualizações.

Recomendação: Recomenda-se realizar um SELECT direto na tabela PRT_PROPSTA e testes de performance para garantir que não haja bloqueios. Priorizar o uso de filtros nas queries para aumentar a velocidade e eficiência. Todas as queries devem passar por um plano de execução e análise de custo com os DBAs de produção. Além disso, considerar a ingestão de dados para o Data Lake para processamentos separados, desonerando a base de dados transacional.

Alessandro Lyra

4816 | DATI - Departamento de Arquitetura de TI